

INHA UNIVERSITY TASHKENT

DEPARTMENT OF CSE & ICE

FALL SEMESTER 2024

SOC 3010 - OPERATING SYSTEMS

HOME ASSIGNMENT 1

INSTRUCTIONS :

- ALL HOME ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 MEMBERS.**
- SCREEN SHOTS ARE TO BE PROVIDED FOR ALL THE QUESTIONS.**
- HOME ASSIGNMENT REPORT SHOULD BE PREPARED USING THE TEMPLATE PROVIDED**
- EVERY MEMBER OF THE TEAM MUST ANSWER ALL THE HOME ASSIGNMENT1 QUESTIONS AND EXECUTE ALL THE PROGRAMS ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS ONLY.**
- SHARING OF QUESTIONS AND SCREESHOTS IS NOT ALLOWED.**
- ONLY PROGRAMS DEVELOPED CAN BE SHARED AMONG THE MEMBERS OF A TEAM**
- A HARDCOPY OF THE TEAM LEADER'S REPORT (CONTAINING ONLY TEAM LEADER'S PART OF THE WORK) MUST BE HANDED IN BY THE TEAM LEADER BY VISITING MY OFFICE ON THURSDAY 26th SEPTEMBER AFTERNOON ANYTIME BETWEEN 2PM & 5PM.**
- EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE IUT ECLASS PORTAL**
- LAST DATE FOR SUBMISSION OF THE HOME ASSIGNMENT1 REPORT AT THE ECLASS PORTAL IS WEDNESDAY 25th SEPTEMBER 2024**
- LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY**
- READ THE QUESTIONS CORRECTLY & CAREFULLY**

ANSWER ALL THE QUESTIONS IN PART I, PART II, PART III, AND PART IV

PART I : UNDERSTANDING UNIX/LINUX COMMANDS

QUESTION [A]

Describe the purpose of the following Unix/Linux commands providing correct syntax. Try out these commands in default bash shell using the correct format and write the results/output obtained and **provide ALL THE SCREEN SHOTS**.

[Hint : use *--help* option after every command or *man* command to understand the command usage and command format, for example, to know the purpose and format of *ls* command - type : `$ ls --help` or `$man ls`]

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE COMMANDS GIVEN HERE ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

- | | | | | | |
|---------------|-------------|---------------|-----------|-----------|-------------|
| 1) ls, ls -la | 2) pwd | 3) cd | 4) cat | 5) cp | 6) mv |
| 7) who | 8) whoami | 9) ps, ps -la | 10) more | 11) less | 12) head |
| 13) tail | 14) fg | 15) rm | 16) mkdir | 17) rmdir | 18) date |
| 19) sudo | 20) apt-get | 21) chmod | 22) echo | 23) find | 24) free |
| 25) diff | 26) grep | 27) passwd | 28) wc | 29) man | 30) sort |
| 31) cmp | 32) bg | 33) file | 34) time | 35) kill | 36) link |
| 37) uname | 38) df | 39) du | 40) chown | 41) chgrp | 42) adduser |
| 43) addgroup | 44) deluser | 45) delgroup | 46) touch | 47) top | 48) cut |
| 49) sed | 50) uptime | | | | |

BE CAREFUL WHEN YOU ARE TRYING OUT THE FOLLOWING REMOVE COMMANDS: 15) rm 17) rmdir FIRST CREATE TEMPORARY FILES AND TEMPORARY DIRECTORIES CONTAINING TEMPORARY FILES AND TRY THESE COMMANDS ON THEM. DO NOT TRY THESE COMMANDS ON YOUR EXISTING FILES AND DIRECTORIES

QUESTION [B]

State the Purpose of the following commands and Indicate the outputs generated when these commands are executed.

EXECUTE THE COMMANDS ONE AFTER ANOTHER IN THE SAME GIVEN ORDER 1) TO 300) AS THE COMMANDS DEPEND ON THE PREVIOUS OPERATIONS.

Clearly explain the format of the output with all the fields and their meaning.

PROVIDE ALL THE SCREEN SHOTS.

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE COMMANDS GIVEN HERE ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

- 1) echo \$PS1
- 2) PS1=" \d \t \u@\h \#\\$"
- 3) ls --help | more
- 4) ls -lt
- 5) ls -lrt
- 6) ls --help | more > lshelp
- 7) man cat
- 8) cat lshelp
- 9) cd ~
- 10) cd .. && ls
- 11) cd && ls
- 12) cat .profile
- 13) uname --help
- 14) uname -r
- 15) uname -s
- 16) uname -n
- 17) uname -o
- 18) uname -p
- 19) uname -a
- 20) man arch
- 21) arch
- 22) man touch
- 23) touch nfact.c
- 24) ls -l nfact.c
- 25) man chmod
- 26) chmod -x nfact.c
- 27) ls -l nfact.c
- 28) cat nfact.c
- 29) man history
- 30) history

31) **history 30**
32) **df --help**
33) **df**
34) **df -h**
35) **df -i**
36) **man top**
37) **top**
38) **man ps**
39) **ps -la**
40) **ps -ef**
41) **ps -ejH**
42) **cat /proc/cpuinfo | more**
43) **cat /proc/cpuinfo > infocpu**
44) **chmod 740 infocpu && ls -l infocpu**
45) **grep --help | more**
46) **grep -e "processor" -e "cpu cores" -e "model name" -e "cpu MHz"**
-e "cache size" infocpu
47) **grep -c "processor" infocpu**
48) **grep -c "cpu" infocpu**
49) **man lsof**
50) **lsof | tee proc_usefile**
51) **man more**
52) **more proc_usefile**
53) **more infocpu**
54) **man head**
55) **head -1 infocpu**
56) **head -5 infocpu**
57) **head -27 infocpu > core0info**
58) **cat core0info**
59) **man tail**
60) **tail -1 infocpu**
61) **tail -5 infocpu**
62) **tail -27 infocpu > core4info**
63) **man cut**
64) **cat core4info**
65) **cut -b1-10 core4info**
66) **cut -b1-25 core4info**
67) **cut -b5-20 core4info**
68) **cut -b15-30 core4info**
69) **man sed**
70) **sed s/cpu/CPU/g core0info**
71) **ls -la | cat > listdir**
72) **cat listdir**
73) **cut -b1-30 listdir**
74) **cut -b30-60 listdir**
75) **history | tail -20**
76) **man uptime**

```
77) uptime
78) du --help
79) du -h
80) du -sh
81) du -sh * | sort -n
82) du -sh * | sort -r
83) man gzip
84) gzip infocpu
85) ls -l info*
86) gunzip infocpu.gz
87) ls -l info*
88) man bzip2
89) bzip2 infocpu
90) ls -la info*
91) bunzip2 infocpu.bz2
92) cat infocpu
93) cat -n infocpu
94) nl --help
95) nl infocpu
96) man pr
97) pr -n infocpu | more
98) grep -o "cpu" infocpu | wc -l
99) man df
100) df -kl
101) df -kh
102) man echo
103) echo -n "HELLO "
104) echo "HOW ARE YOU?"
105) echo $PATH
106) PATH=$PATH:/home/your_login_name (i.e. your login directory)
107) a=$(expr 15 + 10)
108) echo $a
109) let b=75
110) echo $b
111) let c=$a+$b
112) echo $c
113) D=$((15+25))
114) echo $D
115) echo $HOME
116) man find
117) find -name linux
118) ls > list
119) cat list
120) man tee
121) rev list | tee list2
122) cat list2
123) echo "HELLO GOOD MORNING! HOW ARE YOU" | rev
```

124) **man ln**
125) **ln list list1**
126) **ls -l lis***
127) **ln -s list list3**
128) **PS1="[JUNIOR SOC \u@\h\$]"**
129) **ls -l lis***
130) **ls -lrt | grep "^l"**
131) **ls -lrt | grep "^d"**
132) **cat list3**
133) **cat infocpu list1 core0info list2 core4info list3 | more**
134) **man file | tee file.txt**
135) **file list**
136) **file list1**
137) **file list2**
138) **file Desktop**
139) **file *.o**
140) **file *.c**
141) **file *.sh**
142) **file ***
143) **cat core4info**
144) **head -14 core4info | tail -1**
145) **sed -i '1 d' core4info**
146) **cat core4info**
147) **sed -i '\$ d' core4info**
148) **cat core4info**
149) **sed -n '13 p' core4info**
150) **sed -n '13 p' core4info | wc -c**
151) **sed -n '13 p' core4info | wc -w**
152) **sed -n '13 p' core4info | rev**
153) **sed -n '13,18 p' core4info**
154) **sed -n '13,18 p' core4info | wc -l**
155) **sed -n '13,18 p' core4info | wc -w**
156) **sed -n '13,18 p' core4info | wc -c**
157) **sed -n '\$ p' core4info**
158) **echo "GNU is Not Unix" | rev**
159) **echo "GNU is Not Unix" | rev | cut -f1 -d' '**
160) **echo "GNU is Not Unix" | rev | cut -f1 -d' ' | rev**
161) **echo "GNU is Not Unix" | cut -f4 -d' '**
162) **cut -c4 core4info**
163) **cut -c4,6 core4info**
164) **cut -c1-7 core4info**
165) **cut -c-6 core4info**
166) **cut -c10- core4info**
167) **cut -c-10 core4info**
168) **cut -f2 -d' ' core4info**
169) **cut -f1-2 -d' ' core4info**
170) **cut -d' ' -f2 core4info**

- 171) cut -d' ' -f2,3 core4info
- 172) cut -d' ' -f1-3 core4info
- 173) cut -d' ' -f-3 core4info
- 174) cut -d' ' -f2- core4info
- 175) cut -d': ' -f1 /etc/passwd
- 176) cut -d': ' -f2,4 /etc/passwd
- 177) cp /etc/passwd password
- 178) cat password
- 179) cut -d' ' -f1 password
- 180) cut -d' ' -f1 password | cut -c1-10
- 181) rev password | cut -d' ' -f1
- 182) rev password | cut -d' ' -f1 | cut -c10-50
- 183) gedit sample.sh **(Enter the following shell script in the editor & save)**

```
#!/bin/bash
echo Hi, What is your name
read NAME
echo Good Morning $NAME
echo =====
echo Your Home directory : $PWD
echo Enviroment variable PATH = $PATH
echo PRESS Enter KEY
read key
echo =====
echo HARD DISK FREE SPACE INFO
free
echo PRESS Enter KEY
read key
echo =====
echo PROCESSES CURRENTLY RUNNING
ps -la
echo PRESS Enter KEY
read key
echo =====
echo LINUX VERSION
uname -a
echo PRESS Enter KEY
read key
echo =====
echo WHO ARE CURRENTLY LOGGED IN
who -a
echo PRESS Enter KEY
read key
echo =====
read -p 'Login name:' LOGIN
read -sp 'Password:' PASS
echo \n
echo Your LOGIN NAME IS $LOGIN
echo Your PASSWORD IS $PASS
echo =====
echo SIGNING OFF ...GOOD BYE
```

- 184) grep "PRESS" sample.sh
- 185) grep "read" sample.sh
- 186) grep "echo" sample.sh
- 187) vi gptable.txt

(enter the following text in the same order on separate lines in vi editor and save)

GUID Partition Table (GPT) is a standard for the layout of the partition table on a

physical storage device used in a desktop or server PC, such as a hard disk drive or solid-state drive

using globally unique identifiers (GUID). Although it forms a part of the

Unified Extensible Firmware Interface (UEFI) standard (Unified EFI Forum proposed replacement for the PC BIOS), it is also used on some BIOS systems because of the limitations of master boot record (MBR) partition tables, which use

32 bits for storing logical block addresses (LBA) and size information on a traditionally 512-byte disk sector. All modern PC operating systems support GPT. Some, including macOS and

Microsoft Windows on x86, support booting from GPT partitions only on systems with EFI firmware, but

FreeBSD and most Linux distributions can boot from GPT partitions on systems with both legacy BIOS firmware interface and EFI.

- 188) **grep “^[a-z]” gptable.txt**
- 189) **grep “^[A-Z]” gptable.txt**
- 190) **grep “^[0-9]” gptable.txt**
- 191) **grep “^[]” gptable.txt**
- 192) **grep -L “echo” ***
- 193) **grep -L “GPT” ***
- 194) **grep -L “main” ***
- 195) **grep -l “echo” ***
- 196) **grep -l “GPT” ***
- 197) **grep -l “main” ***
- 198) **grep -o “echo” ***
- 199) **grep -o “echo” *.***
- 200) **grep -o “read” *.***
- 201) **grep -n “read” *.***
- 202) **grep -o “echo” sample.sh**
- 203) **grep -n “echo” sample.sh**
- 204) **grep -n “KEY” sample.sh**
- 205) **grep -o “KEY” sample.sh**
- 206) **grep -o “key” sample.sh**
- 207) **grep -o “echo” sample.sh | wc -l**
- 208) **grep -o -b “echo” sample.sh**
- 209) **grep -o -b -n “echo” sample.sh**
- 210) **grep -b -n “echo” sample.sh**
- 211) **grep -b “key” sample.sh**
- 212) **grep -o -b -n “key” sample.sh**
- 213) **grep -b -n “key” sample.sh**
- 214) **grep -b “GPT” gptable.txt**
- 215) **grep -n “key” sample.sh**
- 216) **grep “^echo” sample.sh**
- 217) **grep “KEY\$” sample.sh**


```

218) grep "ey$" sample.sh
219) grep ":$" sample.sh
220) grep "^GUID" gptable.txt
221) grep "^BIOS" gptable.txt
222) grep -v "EFI" gptable.txt
223) grep -v "^echo" sample.sh
224) grep -v "^read" sample.sh
225) grep -c "read" sample.sh
226) grep -c "echo" sample.sh
227) grep -c "file" gptable.txt
228) grep -ow "UEFI" gptable.txt
229) grep -ow "UEFI" gptable.txt | wc -l
230) grep -ow "read" sample.sh
231) grep -ow "read" sample.sh | wc -l
232) cat >> part2q

```

Type this text at the console each on separate lines including this line

For your QUESTION [B] questions - State the Purpose of these commands and Indicate the outputs generated when these commands are executed.

Execute the commands one after another in the same given order 1) to 300).

Clearly explain the format of the output with all the fields and their meaning.

Provide all the screen shots.

Then Press ^D Cntrl D (Cntrl & D keys are to be pressed simultaneously to indicate the end of the file),

```

233) sed -n '3 p' part2q
234) sed -n '3 p' part2q | grep -o "the"
235) sed -n '3 p' part2q | grep -o "the" | wc -l
236) sed -n '1 p' part2q
237) sed -n '1 p' part2q | grep -o "the" | wc -l
238) sed -n '1,4 p' part2q | grep -o "the" | wc -l
239) sed 's/the/THE/' part2q
240) sed 's/THE/the/g' part2q
241) sed 's/the/THE/2' part2q
242) sed 's/THE /the/3' part2q
243) sed 's/the/{&}/' part2q
244) sed 's/the/THE/p' part2q
245) sed -n 's/THE/the/p' part2q
246) sed 's/the/THE/' part2q | sed 's/commands/instructions/'
247) sed -e 's/THE/the/' -e 's/commands/instructions/' part2q
248) sed 's/instructions/commands/' part2q
249) sed '4 s/the/THE/' part2q
250) sed '1,3 s/the/THE/' part2q
251) sed '2,$ s/THE/the/' part2q
252) sed '/commands/ s/the/THE/' part2q

```

- 253) cp part2q part3q
- 254) sed '3 d' part3q
- 255) sed 'p' part3q
- 256) sed '2,\$ d' part3q
- 257) cat part3q
- 258) sed -n '/commands/ p' part2q
- 259) grep 'commands' part2q
- 260) grep -v 'commands' part2q
- 261) grep -w "commands" part2q
- 262) grep -B 2 "State" part2q
- 263) grep -A 2 "State" part2q
- 264) grep -C 2 "State" part2q
- 265) grep -B 2 "and" part2q
- 266) grep -A 2 "and" part2q
- 267) grep -C 2 "and" part2q
- 268) sed -n '/commands/ !p' part2q
- 269) sed '/commands/ a "You need to learn these commands which are very essential" ' part2q
- 270) man cal
- 271) cal
- 272) cal -3
- 273) cal -y
- 274) man passwd
- 275) passwd
- 276) whoami
- 277) man users
- 278) users
- 279) man mail
- 280) mail -s "Test Mail" youremailid@inha.uz (Press Enter key – You get Cc: here enter the other address you want to cc and then press enter and now type your message and after completing the message Press Ctrl D)
- 281) mail -s "Test Mail" youremailid@inha.uz < message.txt (assuming that your message is stored in a file message.txt)
- 282) mail
- 283) man ping
- 284) ping inha.uz (to come out of this command press Ctrl C)
- 285) ping inha.kr (to come out of this command press Ctrl C)
- 286) ping google.com
- 287) ping amazon.com
- 288) whereis linux
- 289) which sample.sh
- 290) man mount
- 291) mount
- 292) man vmstat
- 293) vmstat
- 294) man netstat

295) netstat

296) PATH=\$PATH:/home/your home directory name(for eg. If your login name is aziz your home directory is /home/aziz and the command will be PATH=\$PATH:/home/aziz – that means you added your current directory pathname to PATH variable.

ONCE YOU HAVE DONE THIS THEN THERE IS NO NEED TO ENTER ./ WHENEVER YOU EXECUTE AN EXECUTABLE FILE, FOR EXAMPLE : TO EXECUTE EXECUTABLE FILE hello TYPE \$hello instead of \$./hello)

297) sudo kbdrate

298) sudo kbdrate -r 11 -d 300

299) history > cmdhistory

300) cat cmdhistory | more

QUESTION [C]

ANSWER THE FOLLOWING QUESTIONS AFTER COMPLETING QUESTIONS [A] & [B]:

1. Write a linux command to list all the symbolic links in the current working directory
2. Write a linux command to create an empty file fileop.c in your directory (without using any editor)
3. Give a linux command to make the file fileop.c only readable for the user and no access rights for group and owner
4. How will you find which operating system your system is running on in linux?
5. How will you run a process in background? How will you bring that into foreground and how will you kill that process?
6. How do you know if a remote host is alive or not?
7. Give a linux command to know all the earlier commands entered by you at the terminal.
8. How do you find which process is taking how much CPU?
9. How do you check how much space left on current drive ?
10. What is the difference between ps -ef and ps -a?
11. How do you find how many cpu are there in your system and their details?
12. How do you find zombie process in linux?
13. There is a file somewhere in your system which has the name part2q. Give a linux command to find that file.
14. How do you find whether your system is 32 bit or 64 bit ?
15. How do you find which processes are using a particular file?
16. There is a file part2q which contains words the, Give a linux command to replace all the occurrences of the in the file with THE?
17. You have listdir file created in your directory using question(B). Give a linux command to display second column values from the file.

18. Your application home directory is full? How will you find which directory is taking how much space?
19. How do you find for how long your System is up?
20. Give a linux command to reverse all lines in your file listdir
21. Give a linux command to display processes sorted on CPU usage.
22. How do you get (display) only first 10 lines from file gptable.txt
23. How do you get (display) last 8 lines from file gptable.txt
24. Give atleast two linux commands to compress and atleast two linux commands to decompress files in your directory. **Give examples in each case taking 6 large files to compress & then decompress and compare the sizes of the original and compressed files. State which compression command or algorithm more efficient in reducing the file size.**
25. Give a linux command to display line numbers along with each line of text in a file
26. Give a linux command to assign execute rights to all for the file gptable.txt
27. What does `rm -r *` do?
28. What is the behavioral difference between `cmp` and `diff` commands. Take an example and illustrate.
29. Give two commands in linux to display first line of a file.
30. Give two commands in linux to display last line of a file.
31. What is the command to find maximum memory taking process on your system?
32. What is the command to find hidden files in your current directory?
33. What is the command to find currently running process in your system?
34. What is the command to find remaining disk space on your system?
35. What is the command to count only the number of lines in the file listdir
36. What is the command to count only the number of words in the file listdir
37. What is the command to count only the number of characters in the file listdir
38. Is there a way to erase all files in the current directory, including all its sub-directories, using only one command?
39. What is the use of the `tee` command? Explain with an example
40. What Linux operating system command would you use to display the shell's environment variables?
41. What is the difference between the commands
`ls -la > file1` and `ls -la > file1 2>&1`
42. How do you find the length of 9th line in your file listdir
43. Give a command to delete first line from your file listdir
44. Give a command to delete last line from your file listdir
45. Give a command to get 4th word in a line in file part2q
46. Give a command to get first word in a line in file part2q
47. Give a command to get last word from a line in file part2q
48. Give a command in linux to reverse a string "WINE Is Not windows Emulator"
49. Give a command to delete lines from 1 to 6 in file listdir
50. Give a command to delete lines from 10 to the end of the file listdir

PART II : LINUX KERNEL SOURCE CODE INSTALLATION

QUESTION [A]

Perform installation of Linux Kernel source code into the user directory using the instructions given in your Lecture slides- REFER LINUX KERNEL SOURCE CODE INSTALLATION INSTRUCTIONS PPTS AT THE ECLASS

Describe clearly all the steps in the installation of kernel source code providing all the required Linux commands and indicating clearly the main directory where the Linux kernel source code is installed.

Provide the SCREEN SHOTS of all the steps involved in Linux kernel source code installation.

EVERY MEMBER OF THE TEAM MUST INSTALL LINUX KERNEL SOURCE CODE ON HIS/HER LAPTOP AND INCLUDE ALL THE SCREENSHOTS OF ALL THE STEPS IN THEIR REPORTS.

QUESTION [B]

Understand the organization of the Linux kernel source directory tree clearly by going through the entire directory tree visiting all the sub-directories and provide the complete kernel directory tree (in the form of figure) and list out all the source code files in each of the sub-directories. (Hint : use tree command to get the complete directory structure and draw or put it in

a suitable form to provide a better presentation of the directory structure)

EVERY MEMBER OF THE TEAM MUST PROVIDE THE TREE STRUCTURE IN THE FORM OF DIAGRAMS INCLUDING ALL THE SCREENSHOTS.

PART III : BASH SHELL PROGRAMMING

QUESTION [A]

Using gedit or any other Editor, create the following shell scripts in your user directory, make them executable and run the shell scripts.

USE THE SAME FILE NAMES AS GIVEN AT THE BEGINNING OF EACH SHELL SCRIPT IN RED.

WRITE THE RESULTS/OUTPUTS OBTAINED AND PROVIDE ALL THE SCREEN SHOTS.

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE SHELL SCRIPTS GIVEN HERE ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS INCLUDING THE SCREENSHOTS OF THE SHELL SCRIPTS ENTERED.

1) Sample Shell script

Shell script name : sample.sh

```
#!/bin/bash
echo Hi, What is your name
read NAME
echo Good Morning $NAME
echo =====

echo Your Home directory : $PWD
echo Environment variable PATH = $PATH
echo PRESS Enter KEY
read key
echo =====
```

echo RAM MEMORY SPACE INFO IN MEGA BYTES

free -m

echo PRESS Enter KEY

read key

echo =====

echo HARD DISK USAGE INFO

du -h

echo PRESS Enter KEY

read key

echo =====

echo HARD DISK SPACE INFO

df -h

echo PRESS Enter KEY

read key

echo =====

echo PROCESSES CURRENTLY RUNNING

ps -la

echo PRESS Enter KEY

read key

echo =====

echo LINUX VERSION

uname -a

echo PRESS Enter KEY

read key

echo =====

echo WHO ARE CURRENTLY LOGGED IN

who -a

echo PRESS Enter KEY

read key

echo =====

read -p 'Login name:' LOGIN

read -sp 'Password:' PASS

echo \n

echo Your LOGIN NAME IS \$LOGIN

echo Your PASSWORD IS \$PASS

echo =====

2) File – Line, word and byte count

Shell script name : fcount.sh

```
#!/bin/bash
echo =====
echo -n Enter a filename to find number of characters, words and lines:
read FNAME
cat $FNAME
echo -n No of lines:
wc -l $FNAME
echo -n No of words:
wc -w $FNAME
echo -n No of bytes:
wc -c $FNAME
echo =====
```

3) File Sorting

Shell script name : fsort.sh

```
#!/bin/bash
echo=====
echo ENTER A FILE NAME TO STORE 10 NAMES
read FILE
echo ENTER 10 NAMES ONE BELOW THE OTHER ON SEPARATE
LINES and THEN PRESS ^D
cat >> $FILE
echo PRESS Enter KEY
read key

echo=====+=====
echo your FILE $FILE CONTENTS
cat $FILE
echo PRESS Enter KEY
read key

echo=====
echo FILE SORTING IN ALPHABETICAL ORDER
sort $FILE
echo PRESS Enter KEY
read key

echo+=====
echo FILE SORTING IN REVERSE ALPHABETICAL ORDER
sort -r $FILE
```

4) Examples for Conditional construct if ...then....else ...fi

Shell script name : flist.sh

```
#!/bin/bash
echo=====
echo Do you want directory listing long or short \( enter 1 or 0 \)
read LONG
if [ $LONG -eq 1 ]
then
ls -la
else
ls
fi
```

5) Examples for Loop Constructs

i) while do done construct

Shell script name : sumodd.sh

```
#!/bin/bash
echo =====
A=1
B=0
C=1
echo Computing SUM OF FIRST n ODD INTEGERS
echo ENTER VALUE OF n=
read n
while [ $A -le $n ]
do
B=$(expr $B + $C)
A=$(expr $A + 1)
C=$(expr $C + 2)
done
echo Sum of FIRST $n odd numbers = $B
```

ii) for dodone construct

a) Shell script name : cfiles.sh

```
#!/bin/bash
echo Files in your HOME Directory with extension .c
for FILE in $HOME/*.c
do
echo $FILE
done
```

b) Shell script name : sum100.sh

```
#!/bin/bash
echo COMPUTING SUM OF FIRST 100 INTEGERS
sum=0
for i in {1..100}
do
    sum=$sum+$i
done
echo SUM OF FIRST 100 INTEGERS = $sum
```

iii) until do..... done construct

Shell script name : fact.sh

```
#!/bin/bash
echo =====
A=1
B=1
echo Computing FACTORIAL OF n
echo -n ENTER VALUE OF n=
read n
until [ $A -gt $n ]
do
    B=$(expr $B \* $A)
    A=$(expr $A + 1)
done
echo FACTORIAL OF $n = $B
```

6) Example for Multi-way Branch - caseesac

Shell script name : case.sh

```
#!/bin/bash
#Menu Driven program to perform listed operations
loop=1
while [ $loop -eq 1 ]
do
echo .....
echo .                MENU                .
echo .                .                    .
echo .  1. LIST DIRECTORY CONTENTS        .
echo .  2. SHOW CURRENT WORKING DIRECTORY. .
echo .  3. DISPLAY LINUX VERSION          .
echo .  4. SHOW FREE SPACE ON DISK        .
echo .  5. SHOW WHO ARE LOGGED IN         .
echo .  6. DISPLAY CONTENTS OF A FILE     .
echo .  7. CREATE OR OPEN A FILE          .
echo .  8. COPY A FILE                    .
```

```

echo . 9. RENAME A FILE .
echo . 10. REMOVE A FILE .
echo . 11. QUIT .
echo .....
echo ENTER YOUR CHOICE :
read CH
case $CH in
1)
    ls
    ;;
2)
    pwd
    ;;
3)
    uname -r
    ;;
4)
    free
    ;;
5)
    who -a
    ;;
6)
    echo -n Enter File Name :
    read FILE
    cat $FILE
    ;;
7)
    echo -n Enter File Name :
    read FILE
    gedit $FILE
    ;;
8)
    echo -n Enter Name of the File to be copied from :
    read FILE1
    echo -n Enter Name of the File to be copied to :
    read FILE2
    cp $FILE1 $FILE2
    ;;
9)
    echo -n Enter OLD File Name:
    read FILE1
    echo -n Enter NEW File Name:
    read FILE2
    mv $FILE1 $FILE2

```

```

;;
10)
    echo -n Enter Name of the File to delete:
    read FILE
    echo -n ARE YOU SURE - CONFIRM 1 OR 0 read OK
    if [ $OK -eq 1 ]
    then
        rm $FILE
        echo File $FILE deleted.....OK
    else
        echo File $FILE not deleted.....OK
    fi
;;
11)
    echo QUITTING.....GOOD BYE
    break
;;
*)
    echo INVALID CHOICE - READ MENU CORRECTLY
;;
esac
done

```

7) Example for Select dodone and if ...eliffi constructs

Shell script name : select.sh

```

#!/bin/bash
#A simple Menu System
OPTIONS='ls longls who free linuxversion cat mv cp QUIT'
PS3='Choose an option:'
select CHOICE in $OPTIONS
do
    if [ $CHOICE == ls ]
    then
        ls
    elif [ $CHOICE == longls ]
    then
        ls -la
    elif [ $CHOICE == who ]
    then
        who -a
    elif [ $CHOICE == free ]
    then
        free
    elif [ $CHOICE == linuxversion ]

```

```

then
    uname -a
elif [ $CHOICE == cat ]
then
    echo -n ENTER FILE NAME :
    read FILE
    cat $FILE
elif [ $CHOICE == mv ]
then
    echo -n ENTER OLD FILE NAME :
    read FILE1
    echo -n ENTER NEW FILE NAME :
    read FILE2
    mv $FILE1 $FILE2
elif [ $CHOICE == cp ]
then
    echo -n ENTER NAME OF FILE TO BE COPIED FROM:
    read FILE1
    echo -n ENTER NAME OF FILE TO BE COPIED TO :
    read FILE2
    cp $FILE1 $FILE2
elif [ $CHOICE == QUIT ]
then
    echo BYE ... BYE
    break
fi
done

```

8) Passing parameters using Command Line Arguments

Shell script name : copy.sh

usage syntax : copy file1 file2

```

#!/bin/bash
echo Copying file $1 to $2
cp $1 $2
echo file $1 CONTENTS
cat $1
echo file $2 CONTENTS
cat $2

```

9) Length of a string

Shell script name : len.sh

```
#!/bin/bash
#Show the length of a String
echo Enter a string :
read STR
echo LENGTH OF THE string : ${#STR}
```

10) Use of Special Variables

Shell script name : var.sh

```
#!/bin/bash
echo =====
echo Name of the Bash script - $0
echo How many arguments were passed to the Bash script - $#
echo All the arguments supplied to the bash script - $@
echo The exit status of the most recently run process $?
echo The process ID of the current script - $$
echo User Name of the user running the script - $USER
echo The hostname of the machine the script is running on - $HOSTNAME
echo The number of seconds since the script was started - $SECONDS
echo Current line number in the Bash script : $LINENO
echo Random number returned by the RANDOM variable - $RANDOM
echo VALUES OF ALL ENVIRONMENT VARIABLES SET UP IN THE CURRENT ENVIRONMENT
env | more
echo =====
```

11) Using Arrays

(i) **Shell script name : array.sh**

```
#!/bin/bash
list=(12 67 123 49 88 123 -9 0 456 126)
let I=0
while [ $I -le 9 ]
do
    echo ${list[$I]}
    let I=$I+1
done
```

(ii) **Shell script name : max.sh**

```
#!/bin/bash
list=(12 67 123 49 88 123 -9 0 456 126)
```

```

let I=0
max=${list[0]}
let I=+$I
while [ $I -le 9 ]
do
    if [ ${list[$I]} -gt $max ]
    then
        max=${list[$I]}
    fi
    let I=$I+1
done
echo Maximum element in the list is : $max

```

(iii) **Shell script name : maxlist.sh**

```

#!/bin/bash
echo -n Enter size of the list :
read N
echo Enter a list of $N numbers :
let K=0
while [ $K -lt $N ]
do
    read VAL
    list[$K]=$VAL
    let K=$K+1
done

let I=0
max=${list[0]}
let I=+$I
while [ $I -lt $N ]
do
    if [ ${list[$I]} -gt $max ]
    then
        max=${list[$I]}
    fi
    let I=$I+1
done
echo Maximum element in the list is : $max

```

12) Procedure/Function Invocation

a) Function to compute length of a string

Shell script name : funlen.sh

```
#!/bin/bash
len_str() {
    #Show the length of a String
    echo Enter a string :
    read STR
    L=${#STR}
    return $L
}
print_str(){
    len=$?
    echo Length of the string $STR is $len
}
len_str
print_str
```

b) Function to compute number of lines, words and bytes in a file

Shell script name : funfcnt.sh

```
#!/bin/bash
echo ENTER FILENAME
read fname

lines_in_file() {
    cat $fname | wc -l
}

words_in_file() {
    cat $fname | wc -w
}

bytes_in_file() {
    cat $fname | wc -c
}

num_lines=$( lines_in_file )
echo The file $fname has $num_lines lines in it
num_words=$( words_in_file )
echo The file $fname has $num_words words in it
num_bytes=$( bytes_in_file )
echo The file $fname has $num_bytes bytes in it
```


QUESTION [B]

WRITE SHELL SCRIPTS TO PERFORM THE FOLLOWING OPERATIONS, EXECUTE THEM AND PROVIDE ALL THE SCREEN SHOTS OF THE SHELL SCRIPTS AND RESULTS :

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE SHELL SCRIPTS DEVELOPED ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS INCLUDING THE SCREENSHOTS OF THE SHELL SCRIPTS ENTERED.

- 1) Write a shell script to replace the 6th line in your file `listdir` with a new line "Hi, this new line has been inserted here"
- 2) Linux shell provides the following file test operators that can be used to test various properties associated with a linux file.

Operator	Description	Example
-b file	Checks if file is a block special file; if yes, then the condition becomes true.	if [-b \$file]
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	if [-c \$file]
-d file	Checks if file is a directory; if yes, then the condition becomes true.	if [-d \$file]
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true. Checks if file is a named pipe; if yes, then the condition becomes true.	if [-f \$file]
-p file	Checks if file is readable; if yes, then the condition becomes true.	if [-p \$file]
-r file	Checks if file is writable; if yes, then the condition becomes true.	if [-r \$file]
-w file	Checks if file is executable; if yes, then the condition becomes true.	if [-w \$file]
-x file	Checks if file has size greater than 0; if yes, then condition becomes true.	if [-x \$file]
-s file	Checks if file exists; is true even if file is a directory but exists.	if [-s \$file]
-e file		

Write a Shell script to read a filename from the keyboard and check whether a file exists in your home directory and if it exists then check whether it is a

regular file or empty file, directory or symbolic link or block or character device file or named pipe. Also test whether the file is readable, writeable and executable.

Try out all types of files and provide all SCREENSHOTS.

- 3) Write a shell script to read size of the list n and list of n numbers from the keyboard and store the elements in an array list and then perform the following operations :
 - (i) find and print the maximum element in the list
 - (ii) find and print minimum element in the list
 - (iii) Search for a given key (key is to be read from the keyboard) in the list – if the given key is found in the list then display the message KEY FOUND and print its position in the list, otherwise print NOT FOUND
- 4) Write a shell script to perform the following operations on the file part2q:
 - (i) Substitute all lower case alphabets with uppercase and store the output in file part2ql
 - (ii) Substitute all upper case alphabets with lowercase and store the output in file part2qu
 - (iii) Replace all the occurrences of commands with COMMANDS and replace all words “the’ in the lines containing word “commands” to “THE” and store the output in file part2qr
 - (iv) First copy the contents of file part2q to part2m. Now delete first and last lines of the file part2m
 - (v) Copy the contents of the files part2q, part2ql, part2qu, part2qr and part2m to one single file part2qa (file concatenation operation)
- 5) Unix/Linux provides several commands to perform shutdown, poweroff and reboot as shown below. Write a bash shell script to perform shutdown by providing the following menu driven screen to the user to select an option to perform the selected operation.
 - Halt - Brings the system down immediately**
 - init 0 - Powers off the system using predefined scripts to synchronize and clean up the system prior to shutting down**
 - init 6 - Reboots the system by shutting it down completely and then restarting it**
 - Poweroff - Shuts down the system by powering off**
 - Reboot - Reboots the system**
 - Shutdown - Shuts down the system**

PART IV : PERFORMING FILE OPERATIONS USING UNIX I/O SYSTEM CALLS

QUESTION [A]

Using gedit or any other Editor, create the following c program files (**filenames : filcreate.c, filopen.c, filcopy.c, filrename.c, fildelete.c, filseek.c**) in your directory. Compile the programs using gcc and execute these programs separately.

Write the file operations performed by these programs with suitable comments and UNIX I/O primitives used.

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE FILE OPERATION PROGRAMS GIVEN HERE ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

PROVIDE THE RESULTS AND ALSO ALL the SCREENSHOTS INCLUDING THE SCREENSHOTS OF THE PROGRAMS ENTERED.

1. Program Name : filcreate.c

```
/* Program using UNIX I/O primitives to perform FILE CREATE operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEF_MODE S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

int main()
{
    int fd, i, fsize, nbytes;
    char ch=0, buf[5120], fname[25];
```

```

printf("CREATING A NEW FILE WITH ALL ACCESS RIGHTS TO USER
AND GROUP AND NO EXECUTE ACCESS TO OTHERS\n");
printf("ENTER FILE NAME : ");

scanf("%s", fname);

fd = open(fname,O_CREAT|O_TRUNC|O_WRONLY|O_APPEND,DEF_MODE);

if (fd < 0)

    printf ("cannot create FILE %s \n", fname);
else
{

printf("NOW ENTER YOUR PROGRAM OR TEXT LINE BY LINE- ONCE
YOU FINISH PRESS KEYS Ctrl D together\n");
i=0;

ch=getchar(); /* to remove the last newline character entered*/
while((ch=getchar()) != EOF)
buf[i++]=ch;

fsize=i;
buf[fsize]='\0';
printf("Total characters stored in your file = %d\n", fsize);
write(fd, buf, fsize);
close(fd);
}
}

```

2. Program Name : filopen.c

```

* Program using UNIX I/O primitives to perform FILE OPEN operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>

#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEF_MODE
S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

```

```

int main()

{
int fd, i, fsize, nbytes;
char ch=0,buf[5120], fname[25];


printf("OPENING AN EXISTING FILE\n");
printf("ENTER FILE NAME : ");
scanf("%s", fname);

fd = open(fname, O_RDWR|O_APPEND, DEF_MODE);
if (fd < 0)

    printf ("cannot open FILE %s - does not exist \n", fname);

else
{
printf(" READING YOUR FILE CONTENTS\n");
nbytes=read(fd, buf, sizeof(buf));
buf[nbytes-1]='\0';
printf("CONTENTS OF YOUR FILE %s - size= %d\n",fname, nbytes);

puts(buf);
close(fd);
}
}

```

3. Program Name : filcopy.c

```

/* Program using UNIX I/O primitives to perform FILE COPY operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEF_MODE S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

int main()
{
int fd, fd1, len, i, fsize, nbytes;
char ch=0, cf, buf[5120], fname[25], cname[25];

```

```

printf("COPYING A FILE \n");
printf("ENTER NAME OF CURRENT FILE TO BE COPIED FROM :");
scanf("%s", fname);
printf("ENTER NAME OF NEW FILE NAME TO BE COPIED TO ");
scanf("%s", cname);
fd = open(fname, O_RDONLY, DEF_MODE);
if (fd < 0)
    printf ("cannot open FILE %s - does not exist \n", fname);
else
{
    if(nbytes=read(fd, buf, sizeof(buf)) <0)
        printf("FILE READ ERROR\n");
    else
    {
        fd1 = open(cname, O_CREAT|O_TRUNC|O_WRONLY, DEF_MODE);
        if (fd1 < 0)
            printf("Cannot create New file %s\n", cname);
        else
        {
            len=strlen(buf);
            if( nbytes=write(fd1, buf, len) < 0)
                printf("FILE WRITE ERROR\n");
            else
                printf("FILE %s has been copied to %s successfully OK ..... \n",
                    fname,cname);
            close(fd1);
        }
        close(fd);
    }
}
}

```

4. Program Name : **filrename.c**

```

/* Program using UNIX I/O primitives to perform FILE RENAME operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

```

```

#define DEF_MODE S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

```

```

int main()
{
int fd, fd1, len, i, fsize, nbytes;
char ch=0, cf, buf[5120], fname[25], cname[25];

printf("RENAMING A FILE \n");
printf("ENTER CURRENT FILE NAME:");
scanf("%s", fname);
printf("ENTER NEW FILE NAME :");
scanf("%s", cname);
printf("FILE %s has been renamed to %s OK.....\n", fname, cname);
rename(fname, cname);
}

```

5. Program Name : fildelete.c

```

/* Program using UNIX I/O primitives to perform FILE DELETE operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEF_MODE S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

int main()
{
int fd, fd1, len, i, fsize, nbytes;
char ch=0, cf, buf[5120], fname[25], cname[25];

printf("DELETING A FILE \n");
printf("ENTER FILE NAME :");
scanf("%s", fname);
ch=getchar(); /* to remove the last newline character entered*/
printf("PLEASE CONFIRM -SURE YOU WANT TO DELETE ...PRESS y/n
:");
scanf("%c", &cf);
if (cf == 'y')
{
    unlink(fname);
    printf("FILE %s deleted OK.....\n", fname);
}
else
    printf("FILE %s not deleted OK.....\n", fname);
}

```

```
}
```

6. Program Name : filseek.c

```
/* Program using UNIX I/O primitives to perform FILE SEEK operation */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>

#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEF_MODE
S_IRUSR|S_IWUSR|S_IXUSR|S_IRGRP|S_IWGRP|S_IXGRP|S_IROTH

int main()

{
int fd, i, fsize, nbytes, count;
long int offset;
char ch=0, buf[5120], fname[25];

printf("OPENING AN EXISTING FILE\n");
printf("ENTER FILE NAME : ");
scanf("%s", fname);

fd = open(fname, O_RDWR|O_APPEND, DEF_MODE);
if (fd < 0)

    printf ("cannot open FILE %s - does not exist \n", fname);

else
{
printf("ENTER THE OFFSET BYTES TO BE SKIPPED FROM BEGINNING
OF FILE\n");
scanf("%ld", &offset);
lseek(fd, offset, 0);
printf("ENTER NUMBER OF CHARACTERS TO BE READ FROM OFFSET
%ld POSITION\n",offset);
scanf("%d", &count);
```



```

printf(" READING FROM YOUR FILE %d bytes AT OFFSET %ld\n", count,
offset);
nbytes=read(fd, buf, count);
buf[nbytes-1]='\0';
printf("CONTENTS OF YOUR FILE %s - size= %d\n",fname, nbytes);

puts(buf);
close(fd);
}
}

```

QUESTION [B]

EVERY MEMBER OF THE TEAM MUST EXECUTE MENU DRIVEN PROGRAM DEVELOPED ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

Write a Menu driven 'C' program (**filename : menufileop.c**) to perform the following file operations using Unix I/O primitives :

1. Create a new file by taking the filename from the user and assigning read, write and execute to user and only read access to group and owner. If file already exists then truncate its contents. Provide appropriate error message if there is any problem in file creation.
2. Open an existing file by taking the filename from the user and prompting the user to enter whether file to be opened in Read only mode, Write only mode with Append or Read & Write only mode with Append. Provide appropriate error message if there is any problem in opening a file or filename entered by the user doesn't exist.
3. Read from a file by asking the user to specify the number of bytes to be read from the opened file or entire contents of the file (till EOF). Then display the specified contents of the file read.
4. Write to a file by prompting the user to enter the text (512 bytes or more – several lines or program or script) to be written to the file from the keyboard and ask the user to press Ctrl D once the user completes entering the text.
5. Seek the contents of an opened file by prompting the user to provide the origin(from the beginning or current position or end of the file) and how many bytes to be skipped(offset). After seeking to required position in the file, prompt the user whether user wants to read or write. If read

option is selected, ask the user how many bytes the user wants to read and then display the requested content. If write option is selected then ask the user to enter the text to be written into the file from the keyboard ending with **Cntrl D (Cntrl & D keys are to be pressed simultaneously to indicate the end of the file)**, write to the file starting from the specified position and then display the contents of entire file contents from the beginning.

6. Delete a file by prompting the user to enter the name of the file to be deleted and also further take confirmation from the user willingness to delete. Only after receiving the confirmation, delete the file.
7. Rename a file by asking the user to enter the old(existing) file name and the new file name.
8. Copy a file by prompting the user to enter the name of the file to be copied from and name of the file to be copied to.
9. Quit – signing off from the menu driven screen by closing all the opened files, if any

Compile the program file menufileop.c using gcc and execute the program.

TRY OUT ALL THE MENU-DRIVEN OPTIONS AND PROVIDE THE RESULTS WITH ALL THE SCREENSHOTS INCLUDING THE SCREENSHOTS OF THE PROGRAM ENTERED.
